Week 2 Assignment — 5 Problems

---

1. ATM PIN Length Validator

Scenario

An ATM app must check that a PIN a customer enters is exactly 4 digits long before allowing them to continue, using only the most basic checks.

Task

- Accept a PIN string.
- Get its length using length().
- If the length is not exactly 4, print "Invalid PIN — must be exactly 4 digits."
- Otherwise, print "PIN length OK."
- This one needs no loop at all — just length() and a single if / else.

Suggested method signature(s)

```
void checkPinLength(String pin)
```

Sample Input / Output

Input	Output
"482"	Invalid PIN — must be exactly 4 digits.
"4820"	PIN length OK.

Concepts covered: *length(), if / else, comparison operators — the smallest possible building blocks.*

2 . Word Reversal Encoder

Scenario

The coding club's "mirror text" mini-game reverses every word in a sentence individually while keeping the word order the same, so "hello club" becomes "olleh bulc".

Task

- Accept a sentence (words separated by single spaces).
- Split it into words using split(" ").
- For each word, build its reverse using a loop and StringBuilder.
- Join the reversed words back together with spaces and print the result.

Suggested method signature(s)

```
String reverseEachWord(String sentence)
```

Sample Input / Output

Input	Output
"hello club"	olleh bulc

Concepts covered: *split(), StringBuilder / reverse(), loops, string joining.*

3 . Product Inventory CSV Parser

Scenario

The warehouse team receives inventory updates as CSV lines and needs a quick parser to split each line into fields and print a formatted record.

Task

- Accept a CSV line in the form "ProductName,SKU,Quantity".
- Use split(",") to break it into fields.
- Validate that exactly 3 fields are present; if not, print "Invalid Record".
- Print a formatted record: "Product: ... | SKU: ... | Qty: ...".

Suggested method signature(s)

```
void parseInventoryRecord(String csvLine)
```

Sample Input / Output

Input	Output
"Wireless Mouse,WM-2201,150"	Product: Wireless Mouse | SKU: WM-2201 | Qty: 150
"Wireless Mouse,150"	Invalid Record

Concepts covered: *split(), array length validation, string concatenation, formatted output.*

4 . Library ISBN Normalizer & Validator

Scenario

A library system's book-intake scanner needs to both normalize and validate ISBN-style codes. A valid code is exactly 13 characters: 3 letters (publisher code) + 4 digits (year) + 6 digits (catalog number). Scanned codes sometimes have stray spaces or a mixed-case publisher code.

Task

- Accept a raw code string that may contain leading/trailing spaces.
- Normalize it: trim() the spaces, then uppercase only the first 3 characters using substring() + concatenation — leave the rest untouched.
- Validate: exactly 13 characters after normalization; the first 3 characters are letters; the remaining 10 are digits (use Character.isLetter() / isDigit() in a loop — no regex).
- If valid, build a formatted display line with StringBuilder: "[PUBCODE] YEAR: 20XX | CATALOG: 123456".
- If invalid, print the specific reason: wrong length, non-letter publisher code, or non-digit body.

Suggested method signature(s)

```
String normalizeCode(String raw) + String validateAndFormat(String code)
```

Sample Input / Output

Input	Output
" pen2026004251 "	[PEN] YEAR: 2026 | CATALOG: 004251

Input	Output
"12N2026004251"	Invalid: publisher code must be 3 letters

Concepts covered: *trim(), substring(), string concatenation, Character.isLetter()/isDigit(), StringBuilder, multi-stage validation.*

5 . Stop-Word-Filtered Word Frequency Report

Scenario

The T&P team wants word-frequency analysis of feedback paragraphs, but common filler words ("the", "was", "and", "a", "is") should be excluded so the report highlights meaningful themes instead of function words.

Task

- Accept a paragraph of feedback text, and treat a small fixed list of stop words as filler: {"the","was","and","a","is","of","in"}.
- Normalize it: convert to lowercase, and strip punctuation such as periods and commas using replace().
- Split the cleaned text into words using split("\\s+").
- Skip any word found in the stop-word list.
- Count the frequency of every remaining unique word (a HashMap is fine).
- Print each unique word with its count, sorted by count in descending order. Ties may appear in any order.

Suggested method signature(s)

```
void printFilteredWordFrequency(String feedback)
```

Sample Input / Output

Input	Output
"The mentor was great, the session was great and clear."	great: 2 mentor: 1 session: 1 clear: 1

Concepts covered: *replace(), split() with whitespace pattern, stop-word filtering, frequency counting, sorting by a derived value.*